

Mastering Modern Web Development: Vue 3, Vuex, and Flask-JWT-Extended

Comprehensive Study Guide

Contents

1	Introduction to Modern Frontend Tooling: Vite	2
1.1	Why Vite Over Vue CLI?	2
1.2	Installation and Project Setup	2
1.2.1	Creating a Project	2
1.3	Single File Components (SFC)	2
1.3.1	Why use <code>scoped</code> ?	2
2	Core Vue 3 Concepts	3
2.1	Options API vs Composition API	3
2.2	Props and Events	3
2.3	Vue Router	3
3	State Management with Vuex	4
3.1	Props Drilling Problem	4
3.2	Vuex Solution	4
3.3	Four Pillars	4
3.3.1	Destructuring Example	4
3.4	When to use Vuex	4
4	Backend Authentication: Flask-JWT-Extended	5
4.1	JWT vs Sessions	5
4.2	Key Decorators	5
4.3	Securing Routes	5
5	Common Errors	5

1 Introduction to Modern Frontend Tooling: Vite

1.1 Why Vite Over Vue CLI?

For years, **Vue CLI** (based on Webpack) was the standard for building Vue applications. However, as applications grew, the development server became slow because it had to bundle the entire project before starting.

Vite (French for “fast”) improves this by:

- **Instant Server Start:** No bundling during development.
- **Lightning-Fast HMR:** Updates only changed modules.

1.2 Installation and Project Setup

You need **Node.js** (v16+).

1.2.1 Creating a Project

```
1 npm create vite@latest my-vue-app
```

Recommended setup:

- Framework: Vue
- Add Router, Linter, Prettier

1.3 Single File Components (SFC)

```
1 <template>
2 </template>
```

1.3.1 Why use scoped?

Without **scoped**, styles like `.btn` affect the entire app. Vue solves this using unique attributes like `data-v-xxxx`.

2 Core Vue 3 Concepts

2.1 Options API vs Composition API

1. **Options API:** Structured (data, methods)
2. **Composition API:** Cleaner and modern

2.2 Props and Events

- **Props:** Parent $\rightarrow$ Child (read-only)
- **Events:** Child $\rightarrow$ Parent using `$emit`

2.3 Vue Router

- `<RouterLink>`
- `<RouterView />`
- Lazy loading: `() => import('./View.vue')`

3 State Management with Vuex

3.1 Props Drilling Problem

Passing data through multiple components unnecessarily.

3.2 Vuex Solution

Central store = single source of truth.

3.3 Four Pillars

- State
- Getters
- Mutations (sync only)
- Actions (async)

3.3.1 Destructuring Example

```
1 actions: {  
2   incrementAsync({ commit }) {  
3     setTimeout(() => {  
4       commit('increment');  
5     }, 1000);  
6   }  
7 }
```

3.4 When to use Vuex

- Local UI → use `data()`
- Shared state → use Vuex

4 Backend Authentication: Flask-JWT-Extended

4.1 JWT vs Sessions

JWT = self-contained token (stateless).

4.2 Key Decorators

- `@jwt.user_identity_loader`
- `@jwt.user_lookup_loader`

4.3 Securing Routes

Use:

```
1 Authorization: Bearer <your_token>
```

5 Common Errors

- npm not found → install Node.js
- Module not found → check import paths